TAPE: Task-Aware Evolutionary Expert Pruning under a Fixed Global Budget

HUANG Yuqi
Student ID: 26025288R
Oct 2026

Abstract

Expert pruning reduces the parameter count of Mixture-of-Experts (MoE) models by removing a subset of experts. Existing methods typically keep or drop experts layer by layer according to per-expert importance scores, decide layer capacities separately from expert selection, and rely on signals that do not directly reflect the needs of the target task. We propose TAPE, a task-aware evolutionary pruning method that jointly searches per-layer capacities and expert identities under a fixed global expert budget, and evaluates each complete pruned expert set by its predictive loss on the target tasks, without updating any model weights. Experiments on ERNIE-4.5-21B-A3B-PT and Granite-3.1-1B-A400M-Base show that TAPE consistently outperforms direct scoring baselines on ERNIE and at 75% retention on Granite, and matches the strongest baseline at the most aggressive compression of the smaller model. Ablations show that most of the gain appears only when layer capacities and expert identities are searched jointly, and an analysis of expert selection shows that, at identical layer capacities, TAPE keeps a substantially different and better-performing expert set than the importance ranking does. Code is available at `https://github.com/YuqiHuang2003/Tape`.

Keywords: Mixture-of-Experts; expert pruning; evolutionary computation; multi-objective optimization

1 Introduction

MoE models route each input to a subset of experts, which enlarges model capacity at a modest activated compute cost [1–3], and they have been adopted by large language models such as Mixtral [4] and DeepSeekMoE [5]. Sparse routing reduces per-input computation, but GPU memory must still hold all expert weights simultaneously, limiting deployment on commodity GPUs [6, 7]. Expert pruning removes experts outright to reduce the parameter count and storage footprint [6, 8]. The pruning decision concerns both how many experts to retain in each layer and which experts occupy those slots. Uniform layer-wise pruning fixes the number of slots per layer in advance and then selects experts by score. Non-uniform capacity allocation allows different layers to maintain different numbers of experts, in a spirit similar to layer-wise sparsity allocation in dense-model pruning [9, 10]. However, if the same importance ranking is used within each layer, the reachable expert sets remain constrained by that ranking [11]. These methods focus on either within-layer ranking or between-layer allocation and do not jointly adjust layer capacities and specific experts in a single search.

In addition, expert importance is usually estimated from routing frequency, activation magnitude, or output reconstruction error on calibration samples [6, 7]. These signals measure each expert's contribution to the calibration distribution without taking the predictive loss on the target task into account, so the resulting ranking can hardly reflect what a specific task requires of the expert set. Task-specific pruning methods use target-task data [7, 8], but they either select experts by individual scores or compare combinations only by layer-wise reconstruction error.

Existing methods thus decide layer capacities and expert identities separately and rarely evaluate the resulting pruned network on the target task. EEP [12] evaluates complete pruned models by accuracy, but keeps the same number of experts in every layer, so task feedback cannot change that allocation.

We propose TAPE (Task-Aware Pruning via Evolution), which encodes per-layer capacities and expert identities in one candidate, searches them jointly under an identical total expert count, and evaluates each complete expert set by its predictive loss on the target question-answering tasks. From the perspective of evolutionary computation, TAPE casts expert pruning as a constrained combinatorial multi-objective problem, in line with the broader use of evolutionary search to configure and compose large models [13–15]. Our main contributions are as follows.

1. A joint search space over per-layer capacities and expert identities under an exact global expert budget, with identity swaps, inter-layer capacity transfers, and set crossover that all preserve the budget, so comparisons are not confounded by parameter count.
2. A continuous task-level objective for complete expert sets that combines the cross-entropy on the correct answer with the KL divergence from the original model's option distribution. Unlike accuracy-based fitness, it separates candidates that answer the same questions correctly, and NSGA-II optimizes both tasks without updating model weights.
3. Equal-capacity ablations, a before/after-evolution comparison, and an analysis of keep/drop decisions that show where the gain of joint selection comes from and where task-level search departs from importance rankings.

2 Related Work

2.1 LLM Pruning and Expert Compression

General-purpose LLM pruning mainly targets dense weights. SparseGPT [16] and Wanda [17] prune in one shot without retraining: the former reconstructs layer outputs using approximate second-order information, while the latter scores each weight by the product of its magnitude and the norm of the corresponding input activation. LLM-Pruner [18] removes coupled structural units according to their dependencies and then recovers performance with a small amount of data. OWL [9] further shows that a uniform sparsity ratio across layers is suboptimal and allocates layer-wise sparsity according to activation outliers. These methods operate on weights or structural units and do not exploit the expert-level modularity of MoE models.

Score-based expert compression usually derives the compression scheme from output contribution, usage frequency, or functional similarity. Not All Experts are Equal [8] studies both task-agnostic and task-specific expert pruning. EAC-MoE [7] uses expert selection frequency, REAP [6] combines gate weights with expert output norms, and STUN [19] combines expert-level and unstructured pruning. These methods are cheap to evaluate, but ranking experts directly does not explicitly assess the downstream task performance of the complete pruned expert set.

Other work merges or reconstructs experts, as in Merge, Then Compress [20], HC-SMoE [21], DERN [22], CAMEx [23], NAMEx [24], and diversity-preserving grouping [25], or compresses them through low-rank and shared representations (MoE-SVD [26], D^2-MoE [27], Sub-MoE [28]) and mixed precision [29]. We keep expert weights unchanged and isolate the effect of the retained expert set on task quality.

2.2 Evolutionary Search and Joint Expert Selection

Evolutionary search is widely used to determine non-uniform compression configurations. For dense models, EvoPress [10] uses evolutionary search to choose compression levels for different layers or modules, covering settings such as layer dropping, unstructured sparsity, and quantization. For MoE compression, EEP [12] optimizes expert selection with gradient-free evolution and extends it to routing mappings and expert merging matrices. MoE-I^2 [30] first determines the layer-wise pruning amounts and then performs within-layer genetic search and combination over small layer blocks, and its full pipeline also includes low-rank decomposition and recovery fine-tuning. EvoESAP [11] searches layer-wise capacities under a fixed global budget but keeps the within-layer expert ranking unchanged.

In model merging, Evolutionary Optimization of Model Merging Recipes [14], CycleQD [31], and MERGE^3 [32] explore evolutionary optimization with respect to combination representations, task diversity, and limited evaluation budgets, respectively. Recent work moves this line to the module level. Wu et al. [15] reassemble the Transformer blocks of pretrained LLMs along both depth and width with evolutionary search, and Qiu et al. [33] merge models at the component level through a bi-objective formulation that trades cross-task performance against storage, solved by a surrogate-assisted evolutionary algorithm.

Existing evolutionary approaches to expert pruning fix the layer-wise pruning amount or the within-layer ranking in advance. EEP keeps an equal number of experts per layer and scores configurations by task accuracy, MoE-I^2 fixes layer-wise amounts before selection, and EvoESAP fixes the within-layer ranking. TAPE instead searches binary keep sets over the original experts across the whole model, adjusting capacities and identities within one candidate, and evaluates them with continuous per-task losses.

3 Method

3.1 Joint Representation under a Strict Global Budget

Consider a model with L MoE layers, where layer l contains E_l routed experts. A candidate is represented by a binary matrix M, where $M_{l,e} = 1$ means that expert e in layer l is kept. The capacity of layer l is

$$k_l = \sum_e M_{l,e}.$$

Every candidate satisfies

$$\sum_{l=1}^{L} k_l = B, \qquad k_l^{\min} \le k_l \le k_l^{\max}.$$

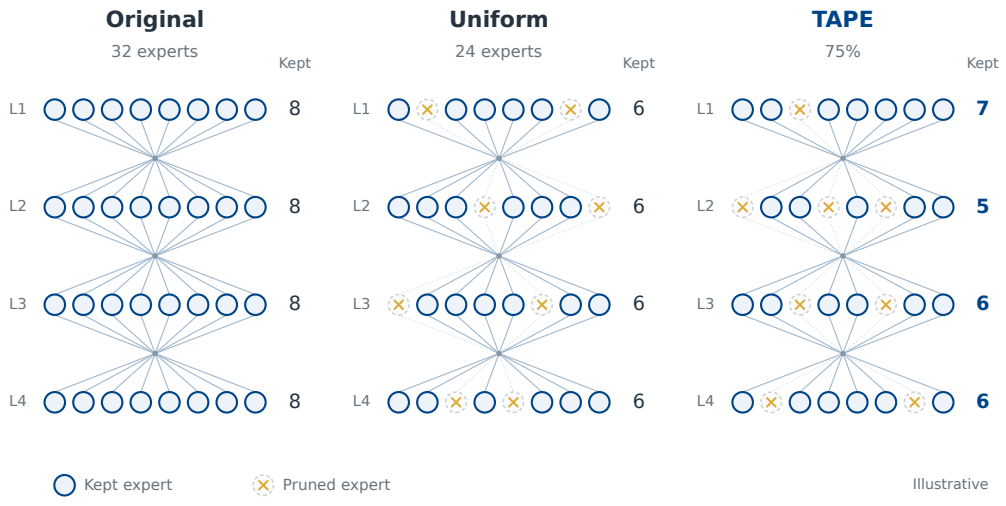


Figure 1: Illustrative comparison of expert pruning at 75% retention. Uniform pruning keeps the same number of experts in every layer, whereas TAPE jointly chooses per-layer capacities and expert identities under the same global budget.

Figure 1 illustrates this representation. To give one layer more slots, a candidate must take the same number from other layers. Candidates are therefore compared on which experts they keep and where

those experts sit, with the total fixed at B. Within each model studied here, all routed experts share the same architecture, so these candidates also retain the same number of expert parameters.

Model weights and router parameters remain frozen throughout the search. The router retains its original top-k setting and selects only among kept experts. Shared experts and all other dense modules remain intact.

3.2 Search Initialization from REAP

The search is initialized from REAP, which scores each expert by its average gate-weighted output magnitude when routed:

$$S_{l,e} = \mathbb{E}\left[g_{l,e}(x)\|f_{l,e}(x)\|_2 \mid e \text{ is selected}\right].$$

TAPE builds the initial population from the REAP solution, which keeps the top-scoring experts in each layer under uniform capacities, together with neighborhood perturbations of it of varying magnitude. These candidates serve only as starting points, and both expert identities and layer-wise slot counts remain free to change jointly under the global budget in subsequent candidates.

3.3 Budget-Preserving Evolutionary Operators

Let $S = \{(l, e) : M_{l,e} = 1\}$ denote the keep set of candidate M, where each element records both the layer index and the expert index, and $|S| = B$. Candidates are generated by the following three types of operators.

An identity swap changes expert selection within a single layer. For $(l, e^-) \in S$ and $(l, e^+) \notin S$,

$$S' = \big(S \setminus \{(l, e^-)\}\big) \cup \{(l, e^+)\}.$$

This operator keeps every layer's capacity unchanged, i.e., $k'_j = k_j$, while allowing combinations of experts that the original importance ranking would never produce.

A capacity transfer removes an expert from one layer and adds an expert to another. For $l \neq m$, $(l, e^-) \in S$, and $(m, f^+) \notin S$,

$$S' = \big(S \setminus \{(l, e^-)\}\big) \cup \{(m, f^+)\}, \qquad k'_l = k_l - 1, \quad k'_m = k_m + 1.$$

A transfer is performed only when $k_l > k_l^{\min}$ and $k_m < k_m^{\max}$. All other layer capacities stay the same, so the global expert count always equals B.

Set crossover recombines two parents, S_a and S_b. Let $C = S_a \cap S_b$ be their common keep set and $D = S_a \triangle S_b$ their disagreement set. Since both parents contain B experts, $|D| = 2(B - |C|)$. We randomly select $|D|/2$ elements of D to form T and generate two complementary offspring

$$\widetilde{S}_1 = C \cup T, \qquad \widetilde{S}_2 = C \cup (D \setminus T).$$

Both offspring contain exactly B experts, $|\widetilde{S}_1| = |\widetilde{S}_2| = B$, but individual layers may fall below their lower bounds or exceed their upper bounds. The repair $S'_j = \mathcal{R}(\widetilde{S}_j)$ redistributes slots until every layer meets its bounds. To keep the total at B, adding a slot to one layer requires removing a slot from another.

Joint mutation combines capacity transfers with identity swaps, so the search can both reallocate slots between layers and change which experts occupy them. To form the next generation, NSGA-II [34] ranks candidates via non-dominated sorting and uses crowding distance to retain solutions that represent different trade-offs between the two tasks.

3.4 Task-Aware Optimization Objective

The search uses a small number of labeled questions from the training split of each target task and does not update model weights. For question x and answer option a_j, the conditional log-likelihood score is

$$s_M(x, a_j) = \frac{1}{c(a_j)} \sum_{u=1}^{|a_j|} \log p_M(a_{j,u} \mid x, a_{j,<u}).$$

Here, $|a_j|$ is the number of tokens in the option. Setting $c(a_j) = 1$ gives its total log-likelihood, and setting $c(a_j)$ to its number of characters gives the length-normalized score. Dividing the scores by a temperature τ and applying a softmax yields the option distribution q_M of the candidate model, and the original model yields p_0 in the same way. The objective for each task t is

$$L_t(M) = \mathbb{E}_{(x,y)\in\mathcal{D}_t} \left[\lambda_{\text{CE}} \big(-\log q_M(y \mid x) \big) + \lambda_{\text{KL}} \, \text{KL}\big(p_0(\cdot \mid x) \| q_M(\cdot \mid x) \big) \right].$$

Here, λ_{CE} and λ_{KL} are the weights of the correct-answer cross-entropy and the teacher KL term, respectively. Both are non-negative and sum to 1. The cross-entropy term measures how well the candidate predicts the task answer, while the teacher KL term limits how far its option distribution deviates from that of the original model, similar to using the teacher's output distribution as a soft target in knowledge distillation [35]. The KL divergence is computed over the options of each question. The scores require only forward passes, and the loss is used to compare candidate expert sets rather than for backpropagation.

The evolutionary search minimizes two separate objectives simultaneously:

$$\min_M \big(L_{\text{ARC}}(M), L_{\text{OBQA}}(M) \big).$$

The two tasks are optimized as separate objectives rather than averaged into a single score, which preserves candidates that favor different tasks. Compared with accuracy on the small search set, the fitness used by EEP [12], the continuous loss provides a finer-grained evaluation signal and distinguishes candidates with identical accuracy.

4 Experimental Setup

4.1 Models and Compression Settings

We use ERNIE-4.5-21B-A3B-PT [36] as the main model and Granite-3.1-1B-A400M-Base [37] for validation on a smaller model. The two models use their default top-6 and top-8 routing, respectively. Expert weights, router weights, and the parameters of all non-pruned modules are kept fixed throughout the experiments.

Table 1 lists the original models and the two pruning budgets. ERNIE has 27 MoE layers with 64 routed experts each, and Granite has 24 MoE layers with 32 routed experts each.[1] The total expert count is the sum of kept experts over all MoE layers, and the expert retention rate refers only to the fraction of routed experts kept. The theoretical total parameter count is computed for the structure obtained after removing the pruned experts and their corresponding router output rows. Other dense modules and shared experts are retained, so the whole-model parameter reduction is smaller than the fraction of experts removed.

For a given model and retention rate, all methods keep exactly the same total number of experts. Scoring baselines use uniform layer-wise capacities, whereas TAPE may reallocate slots between layers within the ranges in Table 1. The total parameter count is computed from the retained structure.

[1] Layer and expert configurations are taken from the official ERNIE config and Granite config released by the model providers.

Table 1: Model sizes and expert retention budgets.

Model	Expert retention	Total experts kept	Theoretical params (B)	Whole-model param reduction	Allowed experts per layer
ERNIE	100%	1,728	21.949	0.00%	64
ERNIE	75%	1,296	16.851	23.22%	44–52
ERNIE	50%	864	11.754	46.45%	28–36
Granite	100%	768	1.335	0.00%	32
Granite	75%	576	1.033	22.63%	20–28
Granite	50%	384	0.731	45.25%	12–20

4.2 Tasks, Data, and Evaluation Metrics

We use two multiple-choice question-answering tasks, ARC-Easy [38] and OpenBookQA [39]. Small subsets of the train split are used for search and solution selection, and no model weights are trained. Final evaluation is performed on the test split, which contains 2,376 questions for ARC-Easy and 500 for OpenBookQA.

The primary metric is length-normalized accuracy (acc_norm). For each question, the conditional log-likelihood of every option is divided by the number of characters in the option text, and the highest-scoring option is compared with the gold answer. This definition matches acc_norm in the LM Evaluation Harness [40] and the metric used in REAP Appendix C [6].

The average (Avg) is the equally weighted mean of the two task accuracies, and the performance retention (Ret) is $\mathrm{Avg}_{\mathrm{pruned}}/\mathrm{Avg}_{\mathrm{original}} \times 100\%$. Scores and differences are computed from unrounded results and reported as percentages to two decimal places.

4.3 Baselines

Original is the unpruned model. REAP, the main baseline, keeps the top experts in each layer by gate-weighted output norm. Frequency ranks experts by how often they are routed on calibration samples. EAN-Mean ranks them by mean output norm when activated, without the gate weight; this conditional-mean variant differs from EAN in REAP Appendix D [6], which uses the cumulative norm, and is reported for ERNIE. REAP+Frequency sums the layer-wise standardized REAP score and standardized log-frequency. All baselines keep the same total number of experts as TAPE.

4.4 Hyperparameters and Implementation Details

All settings use 64 training questions per task for search and 256 disjoint training questions per task for final selection. NSGA-II runs for 15 generations with a population of 16 and 16 offspring per generation (256 candidates), initialized from REAP and its perturbations. Objectives use length-normalized option scores with $\tau = 0.25$, $\lambda_{\mathrm{CE}} = 0.75$, and $\lambda_{\mathrm{KL}} = 0.25$. Final selection compares all 16 initial candidates and the 8 later candidates with the highest search-set mean acc_norm, using mean two-task acc_norm on the additional training questions. Expert sets are fixed before test evaluation. Main experiments and both search ablations use three independent seeds, with means and sample standard deviations reported. The w/o Evolved Identities ablation and the evolution and expert-selection analyses use one ERNIE 75% run. All experiments use a single NVIDIA A100-SXM4-80GB GPU with BF16 inference and no model-weight updates.

5 Results and Analysis

5.1 Main Results on ERNIE

Table 2 reports test results under the search setting of Section 4.4. The TAPE solutions are determined by the training-set selection rule in Section 4.4.

At 75% retention, TAPE reaches a mean Avg of 62.27%, 3.77 percentage points higher than REAP and 4.13 points higher than EAN-Mean, and all three seeds exceed every baseline. Relative to REAP,

Table 2: Test results on ERNIE at two expert retention rates. TAPE reports the mean ± standard deviation over three search seeds, and the deterministic baselines are evaluated once.

Expert retention	Method	ARC-Easy Test(%)	OpenBookQA Test(%)	Avg(%)	Ret (%)
100%	Original	78.24	46.20	62.22	100.00
75%	REAP	75.80	41.20	58.50	94.02
75%	Frequency	71.38	37.20	54.29	87.25
75%	EAN-Mean	74.07	42.20	58.14	93.44
75%	REAP+Frequency	73.23	38.40	55.82	89.71
75%	TAPE	80.86 ± 0.28	43.67 ± 0.70	62.27 ± 0.35	100.07 ± 0.56
50%	REAP	63.34	34.20	48.77	78.38
50%	Frequency	59.30	29.20	44.25	71.12
50%	EAN-Mean	67.42	32.60	50.01	80.38
50%	REAP+Frequency	61.95	34.00	47.98	77.11
50%	TAPE	68.76 ± 0.89	33.60 ± 0.72	51.18 ± 0.77	82.25 ± 1.24

it improves ARC-Easy and OpenBookQA by 5.06 and 2.47 points, respectively. With a 23.22% reduction in theoretical total parameters, TAPE matches the original model, with a performance retention of 100.07%. At the task level, ARC-Easy is 2.62 points above the original model, and OpenBookQA is 2.53 points below.

At 50% retention, TAPE reaches a mean Avg of 51.18%, 2.41 points higher than REAP and 1.17 points higher than EAN-Mean, the strongest baseline at this budget. Relative to EAN-Mean, it improves ARC-Easy and OpenBookQA by 1.34 and 1.00 points, respectively. With a 46.45% reduction in theoretical total parameters, it retains 82.25% of the original model's average score. The gain from continued evolution at 75% retention is analyzed in Section 5.4.

5.2 Compression Results on Granite

Table 3: Test results on Granite at two expert retention rates. TAPE reports the mean ± standard deviation over three search seeds, and the deterministic baselines are evaluated once.

Expert retention	Method	ARC-Easy Test(%)	OpenBookQA Test(%)	Avg(%)	Ret (%)
100%	Original	67.59	38.00	52.80	100.00
75%	REAP	46.89	30.60	38.74	73.38
75%	Frequency	52.69	30.40	41.55	78.69
75%	REAP+Frequency	53.45	32.80	43.13	81.68
75%	TAPE	54.83 ± 0.69	33.47 ± 0.90	44.15 ± 0.38	83.61 ± 0.72
50%	REAP	33.46	26.80	30.13	57.07
50%	Frequency	28.49	29.00	28.75	54.45
50%	REAP+Frequency	35.90	28.00	31.95	60.52
50%	TAPE	37.29 ± 0.76	27.07 ± 0.12	32.18 ± 0.42	60.94 ± 0.79

At 75% retention, TAPE reaches a mean Avg of 44.15%, 5.41 points higher than REAP and 1.02 points higher than REAP+Frequency, the strongest direct scoring baseline, and all three seeds exceed every baseline. It retains 83.61% of the original average score. At 50% retention, TAPE reaches a mean Avg of 32.18%, 2.05 points higher than REAP and on par with REAP+Frequency, with a performance retention of 60.94%. The two methods differ in how they balance the tasks. Across all seeds, TAPE scores higher than REAP+Frequency on ARC-Easy, by 1.39 points on average, and lower on OpenBookQA, by 0.93 points on average.

ERNIE shows higher performance retention than Granite at both budgets. With only 32 experts per layer, Granite's performance retention drops from 83.61% to 60.94% when compressed from 75% to 50% retention, indicating greater sensitivity to compression. At 50% retention, the comparison

with REAP+Frequency also exposes a trade-off between ARC-Easy and OpenBookQA that a single average score hides.

5.3 Ablation Study

The ablations are conducted on ERNIE at 75% expert retention. TAPE and the two search ablations are each reported as the mean over three search seeds. w/o Layer Allocation fixes 48 experts per layer and searches only expert identities. w/o Identity Search searches layer capacities and selects experts by the fixed REAP ranking. Both variants are searched independently, initialized under their own constraints, and use the same sample sizes, candidate budget, and selection rule as the main experiment. w/o Evolved Identities fixes the layer-wise capacities of the TAPE solution used for the analyses in Sections 5.4 and 6 and reselects experts by the REAP ranking, which compares expert combinations at identical capacities.

Table 4: Ablation results on ERNIE at 75% expert retention. TAPE, w/o Layer Allocation, and w/o Identity Search report the mean over three search seeds. w/o Evolved Identities is evaluated once, on the capacities of the TAPE solution analyzed in Sections 5.4 and 6.

Method	ARC-Easy Test(%)	OpenBookQA Test(%)	Avg(%)
TAPE	80.86	43.67	62.27
w/o Layer Allocation	77.99	41.60	59.79
w/o Identity Search	76.43	40.40	58.42
w/o Evolved Identities	76.14	40.00	58.07

TAPE outperforms w/o Layer Allocation by 2.48 points and w/o Identity Search by 3.85 points on average. With the same sample sizes and candidate budget, neither restricted search recovers the gain of the joint search. Searching only layer capacities gives an Avg of 58.42%, comparable to the 58.50% of uniform-capacity REAP, showing that reallocating slots between layers provides little benefit when the within-layer ranking is fixed. Searching only expert identities raises Avg to 59.79%, 1.29 points above REAP. The two restricted searches together improve on REAP by 1.21 points, whereas the joint search improves on it by 3.77 points. Identity search is the more useful of the two on its own, but most of the gain appears only when capacities and identities are searched together: the extra slots given to a layer pay off once the search can also choose which experts fill them.

With the layer capacities of a TAPE solution fixed, selecting experts by the REAP ranking gives an Avg of 58.07%, slightly below uniform-capacity REAP (58.50%) and 4.60 points below that TAPE solution (62.67%), which has exactly the same capacities and differs only in which experts are kept. The TAPE capacities are thus useful only together with the identities selected for them.

5.4 Effect of Evolutionary Search

To measure the gain of evolutionary search over initialization, we compare TAPE after initialization only (generation 0) with TAPE after the full 15 generations. The generation-0 control selects a solution from the 16 initial candidates alone, while the full search also includes the later candidates specified in Section 4.4. Both use the same additional training questions and final selection rule, and test acc_norm is evaluated after the solution is fixed. Both rows come from the same search run, which is one of the three runs included in the main results of Section 5.1.

Table 5: Test results of TAPE before and after evolution on ERNIE at 75% expert retention.

Solution	ARC-Easy Test(%)	OpenBookQA Test(%)	Avg(%)
Before evolution (gen 0)	80.60	41.60	61.10
After evolution (gen 15)	80.93	44.40	62.67

The solution finally selected by the full search was produced in generation 14. Compared with the solution selected by the generation-0 control, it changes the capacities of 16 layers and replaces 26

experts, raising Avg from 61.10% to 62.67%, an increase of 1.57 points. ARC-Easy improves by 0.34 points and OpenBookQA by 2.80 points, so most of the gain comes from the latter. Continued evolution finds better expert combinations that were absent from the initial population.

6 Expert Selection Behavior

Building on the equal-capacity comparisons in Section 5, this section analyzes how TAPE and importance rankings differ in their keep/drop decisions and how these choices relate to task loss, examining whether higher-scoring experts are also more beneficial to the target tasks.

6.1 Keep/Drop Differences from REAP

We take the ERNIE 75% solution of Section 5.4, which keeps 1,296 of 1,728 experts. Against uniform-capacity REAP, both keep 1,179 and remove 315, and TAPE re-keeps 117 experts and newly removes 117, so 72.9% of its removals agree with REAP. To remove the effect of capacity, we construct an equal-capacity REAP set using TAPE's layer-wise slot counts. TAPE then keeps 128 experts that the REAP ranking would remove and removes 128 that it would keep, across all 27 MoE layers (Figure 2), with test averages of 62.67% and 58.07%, respectively.

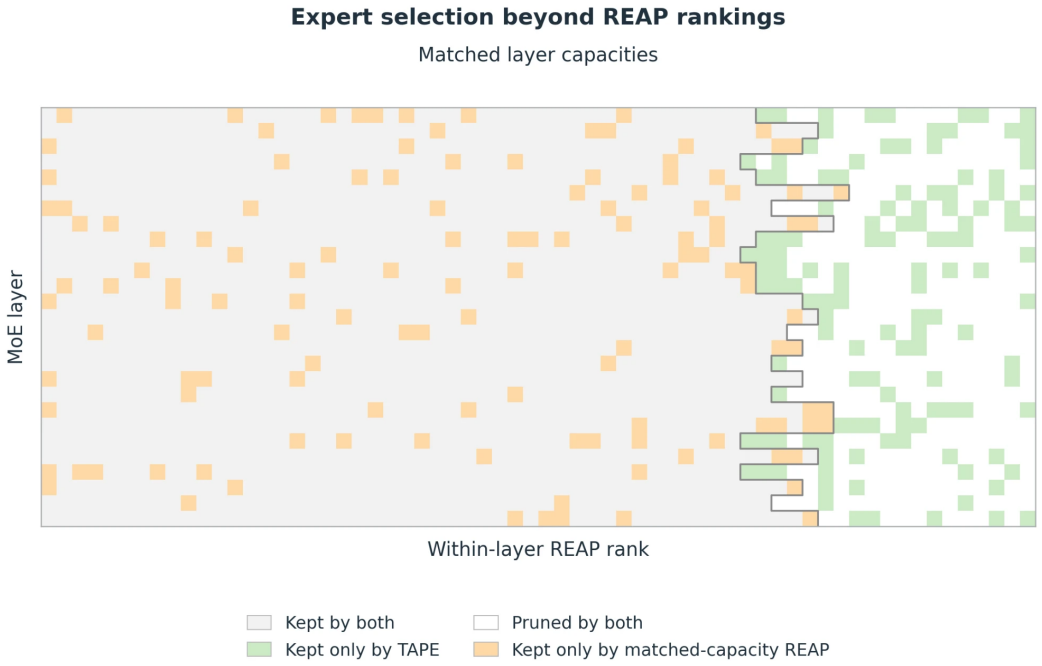


Figure 2: Expert keep/drop decisions of TAPE and matched-capacity REAP on ERNIE at 75% expert retention, with experts in each layer ordered by REAP rank. The outline marks the matched-capacity REAP cut-off.

Aligning capacities does not reduce the difference; the number of differing experts in each direction even grows from 117 to 128. TAPE and REAP therefore disagree mainly about which experts to keep in each layer rather than about how slots are allocated, and these substitutions are spread across all layers.

6.2 Association between Expert Selection and Task Performance

For the 256 candidates of this run, we split candidates by whether a given expert is kept and compare their task objectives, standardized and linearly adjusted for generation and layer capacity (both groups must contain at least 26 candidates). The retention association is the mean adjusted objective

of the removal group minus that of the keep group, so a positive value means keeping the expert is associated with lower task loss. Table 6 lists the two re-kept experts with the highest mean association and the two newly removed experts with the lowest.

Table 6: Task retention associations of selected experts in the equal-capacity comparison.

Expert	Layer slots	REAP rank in layer	TAPE decision	ARC retention association	OBQA retention association
L11-E20	46	50 / 64	Keep	$+0.113$	$+0.147$
L21-E46	51	53 / 64	Keep	$+0.100$	$+0.085$
L21-E59	51	50 / 64	Remove	-0.176	-0.156
L25-E15	49	13 / 64	Remove	-0.103	-0.157

Consider layer L21, which keeps 51 experts. Under the REAP ranking, E59 (ranked 50th) would be kept and E46 (ranked 53rd) would be removed, whereas the final TAPE set does the opposite. Across the search population, candidates that keep E46 have lower loss on both tasks and candidates that keep E59 have higher loss, so for this pair the task signal runs opposite to the REAP order. The disagreement is not limited to experts near the cut-off. L25-E15 ranks 13th of 64 in its layer, well inside the REAP keep set, yet candidates that drop it have lower loss on both tasks. Because TAPE compares complete expert sets by task loss, it can follow these signals instead of the order of per-expert scores.

7 Discussion and Limitations

The results point to a clear trade-off between cost and quality. Direct scoring is nearly free once calibration statistics are collected, whereas TAPE evaluates 256 candidates with forward passes on 64 questions per task. On a single A100, this search takes under 5 minutes for Granite and 75–106 minutes for the ERNIE runs with recorded timing (Appendix A). In return, TAPE reaches expert sets excluded by the fixed REAP ranking, and its gain over the strongest-scoring baseline is largest at 75% retention across both models.

The search in TAPE relies on a small number of labeled examples from the target tasks and builds its loss on the option distribution of multiple-choice questions. Tasks such as open-ended text generation have no fixed answer candidates and require a different task loss. Our ablations vary the search space while keeping the task-level objective fixed; running the same search with a task-agnostic objective, such as language-modeling loss on generic text, would measure the objective's contribution on its own. Future work can extend TAPE to more question-answering and generation tasks and evaluate pruned models on tasks not used in the search, to study how well task-aware expert selection transfers across tasks.

Each candidate requires a full forward pass over the search examples, so the cost grows with model size; for MoE models with hundreds of billions of parameters it exceeds the resources of this project. Surrogate models or computation reuse across candidates could reduce this cost. Finally, a mechanistic study of expert specialization, swapping experts one at a time at fixed capacities across many architectures, would show whether specialization and pruning sensitivity follow common patterns across models.

8 Conclusion

We proposed TAPE, a task-aware evolutionary method for MoE expert pruning under a fixed expert budget that jointly searches per-layer capacities and expert identities with budget-preserving operators and evaluates complete expert sets by task loss, using forward passes only. On two MoE models, TAPE outperforms direct scoring in most settings and matches the original model on ERNIE at moderate compression. Most of the gain requires the joint search, and highly ranked experts are not necessarily the ones the task needs. Under a fixed budget, an expert's value depends on the combination it belongs to, so complete expert sets should be evaluated on the target task rather than assembled from per-expert rankings.

References

[1] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

[2] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2006.16668.

[3] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

[4] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. URL https://arxiv.org/abs/2401.04088.

[5] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1280–1297, 2024. doi: 10.18653/v1/2024.acl-long.70. URL https://aclanthology.org/2024.acl-long.70/.

[6] Mike Lasby, Ivan Lazarevich, Nish Sinnadurai, Sean Lie, Yani Ioannou, and Vithursan Thangarasa. REAP the experts: Why pruning prevails for one-shot MoE compression. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/ed93b2b5722acc2341d421b8916404a1-Abstract-Conference.html.

[7] Yuanteng Chen, Yuantian Shao, Peisong Wang, and Jian Cheng. EAC-MoE: Expert-selection aware compressor for mixture-of-experts large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12942–12963, 2025. doi: 10.18653/v1/2025.acl-long.633. URL https://aclanthology.org/2025.acl-long.633/.

[8] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6159–6172, 2024. doi: 10.18653/v1/2024.acl-long.334. URL https://aclanthology.org/2024.acl-long.334/.

[9] Lu Yin, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Gen Li, Ajay Kumar Jaiswal, Mykola Pechenizkiy, Yi Liang, Michael Bendersky, Zhangyang Wang, and Shiwei Liu. Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 57101–57115, 2024. URL https://proceedings.mlr.press/v235/yin24e.html.

[10] Oliver Sieberling, Denis Kuznedelev, Eldar Kurtic, and Dan Alistarh. EvoPress: Accurate dynamic model compression via evolutionary search. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 55556–55590, 2025. URL https://proceedings.mlr.press/v267/sieberling25a.html.

[11] Zongfang Liu, Shengkun Tang, Boyang Sun, Zhiqiang Shen, and Xin Yuan. EvoESAP: Non-uniform expert pruning for sparse MoE. *arXiv preprint arXiv:2603.06003*, 2026. URL https://arxiv.org/abs/2603.06003.

[12] Enshu Liu, Junyi Zhu, Zinan Lin, Xuefei Ning, Matthew B. Blaschko, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Efficient expert pruning for sparse mixture-of-experts language models: Enhancing performance and reducing inference costs. *arXiv preprint arXiv:2407.00945*, 2024. URL https://arxiv.org/abs/2407.00945.

[13] Xingyu Wu, Sheng-hao Wu, Jibin Wu, Liang Feng, and Kay Chen Tan. Evolutionary computation in the era of large language model: Survey and roadmap. *IEEE Transactions on Evolutionary Computation*, 29(2):534–554, 2025. doi: 10.1109/TEVC.2024.3506731. URL https://arxiv.org/abs/2401.10034.

[14] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 7(2):195–204, 2025. doi: 10.1038/s42256-024-00975-8. URL https://www.nature.com/articles/s42256-024-00975-8.

[15] Xingyu Wu, Yu Zhou, and Kay Chen Tan. Building LLMs like LEGO: Two-dimensional architecture reassembly of large language models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 44939–44956, 2026. doi: 10.18653/v1/2026.acl-long.2081. URL https://aclanthology.org/2026.acl-long.2081/.

[16] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 10323–10337, 2023. URL https://proceedings.mlr.press/v202/frantar23a.html.

[17] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.11695.

[18] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 21702–21720, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/44956951349095f74492a5471128a7e0-Abstract.html.

[19] Jaeseong Lee, Seung-won Hwang, Aurick Qiao, Daniel F. Campos, Zhewei Yao, and Yuxiong He. STUN: Structured-then-unstructured pruning for scalable MoE pruning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13660–13676, 2025. doi: 10.18653/v1/2025.acl-long.671. URL https://aclanthology.org/2025.acl-long.671/.

[20] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. Merge, then compress: Demystify efficient SMoE with hints from its routing policy. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/3d09a88c3372cdb79401fde592ca4db8-Abstract-Conference.html.

[21] I-Chun Chen, Hsu-Shen Liu, Wei-Fang Sun, Chen-Hao Chao, Yen-Chang Hsu, and Chun-Yi Lee. Retraining-free merging of sparse MoE via hierarchical clustering. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 8594–8620, 2025. URL https://proceedings.mlr.press/v267/chen25aq.html.

[22] Yixiao Zhou, Ziyu Zhao, Dongzhou Cheng, Zhiliang Wu, Jie Gui, Yi Yang, Fei Wu, Yu Cheng, and Hehe Fan. Dropping experts, recombining neurons: Retraining-free pruning for sparse mixture-of-experts LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15169–15186, 2025. doi: 10.18653/v1/2025.findings-emnlp.820. URL https://aclanthology.org/2025.findings-emnlp.820/.

[23] Dung Viet Nguyen, Minh Nguyen, Luc Nguyen, Rachel Teo, Tan Nguyen, and Duy Linh Tran. CAMEx: Curvature-aware merging of experts. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/341d64b067e06c6ebea97a498c90d598-Abstract-Conference.html.

[24] Dung V. Nguyen, Anh T. Nguyen, Minh H. Nguyen, Luc Q. Nguyen, Shiqi Jiang, Ethan Fetaya, Linh Duy Tran, Gal Chechik, and Tan M. Nguyen. Expert merging in sparse mixture of experts with Nash bargaining. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/ef9f16cd0cb8d7797ea35c0ee7964cdf-Abstract-Conference.html.

[25] Zeliang Zhang, Xiaodong Liu, Hao Cheng, Chenliang Xu, and Jianfeng Gao. Diversifying the expert knowledge for task-agnostic pruning in sparse mixture-of-experts. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 86–102, 2025. doi: 10.18653/v1/2025.findings-acl.4. URL https://aclanthology.org/2025.findings-acl.4/.

[26] Wei Li, Lujun Li, Hao Gu, You-Liang Huang, Mark G. Lee, Shengjie Sun, Wei Xue, and Yike Guo. MoE-SVD: Structured mixture-of-experts LLMs compression via singular value decomposition. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 35209–35230, 2025. URL https://proceedings.mlr.press/v267/li25az.html.

[27] Hao Gu, Wei Li, Lujun Li, Qiyuan Zhu, Mark G. Lee, Shengjie Sun, Wei Xue, and Yike Guo. Delta decompression for MoE-based LLMs compression. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 20497–20514, 2025. URL https://proceedings.mlr.press/v267/gu25c.html.

[28] Lujun Li, Qiyuan Zhu, Jiacheng Wang, Xiaoyu Qin, Wei Li, Hao Gu, Sirui Han, and Yike Guo. Sub-MoE: Efficient mixture-of-expert LLMs compression via subspace expert merging. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 22994–23002, 2026. doi: 10.1609/aaai.v40i27.39464. URL https://ojs.aaai.org/index.php/AAAI/article/view/39464.

[29] Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. Mixture compressor for mixture-of-experts LLMs gains more. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/abc1943857a42935ceacff03c524bb44-Abstract-Conference.html.

[30] Cheng Yang, Yang Sui, Jinqi Xiao, Lingyi Huang, Yu Gong, Yuanlin Duan, Wenqi Jia, Miao Yin, Yu Cheng, and Bo Yuan. MoE-I^2: Compressing mixture of experts models through inter-expert pruning and intra-expert low-rank decomposition. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10456–10466, 2024. doi: 10.18653/v1/2024.findings-emnlp.612. URL https://aclanthology.org/2024.findings-emnlp.612/.

[31] So Kuroki, Taishi Nakamura, Takuya Akiba, and Yujin Tang. Agent skill acquisition for large language models via CycleQD. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/755acd0c7c07180d78959b6d89768207-Abstract-Conference.html.

[32] Tommaso Mencattini, Robert Adrian Minut, Donato Crisostomi, Andrea Santilli, and Emanuele Rodolà. MERGE^3: Efficient evolutionary merging on consumer-grade GPUs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 43694–43715, 2025. URL https://proceedings.mlr.press/v267/mencattini25a.html.

[33] Haiyun Qiu, Xingyu Wu, Liang Feng, and Kay Chen Tan. Fine-grained model merging via modular expert recombination. *arXiv preprint arXiv:2602.06552*, 2026. URL https://arxiv.org/abs/2602.06552.

[34] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002. doi: 10.1109/4235.996017.

[35] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL https://arxiv.org/abs/1503.02531.

[36] Baidu ERNIE Team. ERNIE 4.5 technical report. https://ernie.baidu.com/blog/publication/ERNIE_Technical_Report.pdf, 2025.

[37] IBM Granite Team. Granite-3.1-1B-A400M-Base model card, 2024. URL https://huggingface.co/ibm-granite/granite-3.1-1b-a400m-base.

[38] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL https://arxiv.org/abs/1803.05457.

[39] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, 2018. doi: 10.18653/v1/D18-1260. URL https://aclanthology.org/D18-1260/.

[40] Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, et al. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*, 2024. URL https://arxiv.org/abs/2405.14782.

A Compute Cost

Table 7 reports, for each run on a single A100-80GB GPU, the wall-clock time of the search stage (256 candidates) and the time of the full test evaluation of the final solution. The search time excludes model loading, teacher-score preparation, final selection on the additional training questions, and test evaluation.

Table 7: Compute cost on a single A100-80GB GPU.

Run	Search (min)	Final test (s)
Granite 75%	4.93	100.54
Granite 50%	3.88	83.16
ERNIE 75%	–[†]	282.90
ERNIE 50%	75.01	210.25
ERNIE 75% w/o Layer Allocation	105.93	288.82
ERNIE 75% w/o Identity Search	103.40	283.48

[†] The search-stage timer of this run was overwritten when the run was resumed from its cache, so no complete search time is available.